

05 Dynamics

BY NILUSHANAN KULASINGHAM

11 MIN READ · INTERACTIVE

A model trained on the truth at every step gets its own last answer instead the moment you run it. What carries the past forward, and why the headline accuracy number measures the wrong job.

A model that predicts one step ahead is easy to be impressed by.

Show it where things are, ask what happens next, compare its answer to what happened. Do that ten thousand times and read off the average error. The number will be small, and it will be honest, and it will not tell you what you want to know.

Because you were never going to use it one step at a time. You were going to ask it for a hundred steps, and on step two it stops being given the truth and starts being given its own answer.

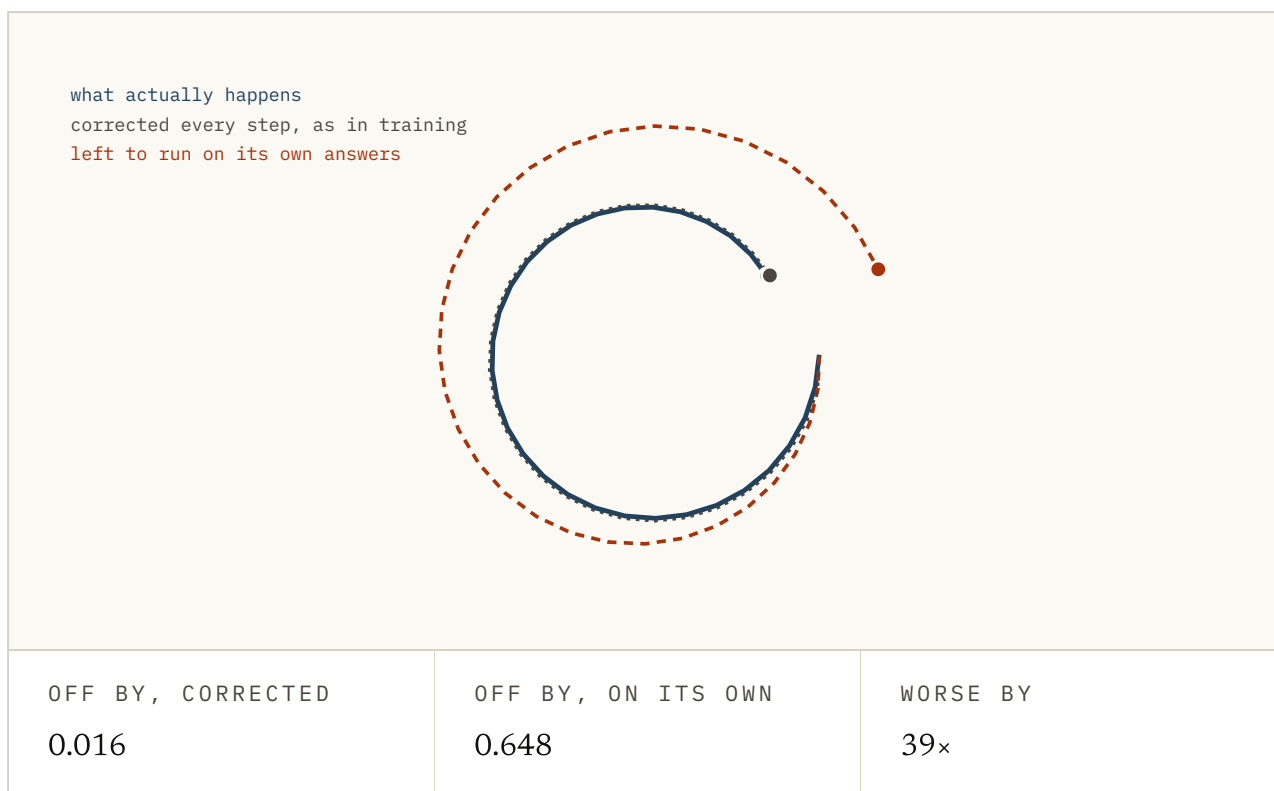


FIG. 5.1 One model, one bias, two ways of running it. The grey line is corrected at every step, the way it was during training, and it tracks the truth so closely that it mostly disappears underneath it. That is the result. The vermilion line is the same model left to consume its own output, the way it will actually be used.

At four per cent error per step, corrected, it is still hugging the truth after thirty steps. Left alone, it ends up thirty-nine times further away. Same model. Same mistake. The only thing that changed is what it was handed at the start of each step.

The mismatch

This has a name, and the name is unhelpfully technical for something so simple. During training, a sequence model is usually shown the true previous value at every step, because that is what the recording contains and it makes the training stable. That is called **teacher forcing**.

The trouble is that nobody teacher-forces you in production. At the moment of use the model gets its own last answer, which is slightly wrong, and it has never once been asked to recover from being slightly wrong.

So it is not that the model is bad at long horizons. It is that it was trained on a distribution of inputs it will never see again the moment you start rolling it out, and nothing in the training loop noticed.

Look at that figure again and notice what the grey line is really telling you. It is not a measure of how good the model is. It is a measure of how good the model is at a job it will never be asked to do.

What has to happen at each step

Strip the machinery away and a transition model does one thing: take what you know, take what you did, and produce what you know next.

The awkward part is the first of those. What you know is not just the last picture. It is everything relevant that has ever happened, folded into something small enough to carry. A ball behind a wall is still moving. A door you opened four rooms ago is still open.

So the real question is how the past gets carried, and there are two answers.

Carry a summary. Keep one fixed block of numbers. Every time something new arrives, fold it in and throw the old block away. The work per step never changes, however long the sequence gets. This is what recurrent networks do (a network with a loop in it: each step's output is handed back as part of the next step's input), and what the state-space models that have come back into fashion do too.

Keep everything. Store every step, and when a new one arrives, look back over all of them and decide what is relevant. Nothing gets thrown away, because nothing has to be summarised. This is attention (for each new step, score every earlier step for

relevance, then read mostly from the ones that scored highest), and it is what transformers do.

	CARRY A SUMMARY	KEEP EVERYTHING
NUMBERS CARRIED FORWARD What the model is holding when the next step arrives.	256	32,768
WORK FOR ONE MORE STEP Multiply-adds to fold in one new observation.	65,536	32,768
CAN IT STILL SEE THE FIRST STEP? Whatever a fixed summary dropped is not recoverable later.	Only if the summary kept it	Yes, exactly, always
BOTH COLUMNS USE A STATE 256 NUMBERS WIDE, SO THE ONLY THING CHANGING IS THE LENGTH.		

FIG. 5.2 Both columns use a state of the same width, so the only thing changing is how long the sequence has got. Neither column is the winner. Notice where the second row crosses over: looking at everything is genuinely cheaper for short sequences, and genuinely not for long ones.

The trade underneath is not really about speed. A fixed summary has to decide, at every step, what is worth keeping, and it has to decide without knowing what will be needed later. Attention never has to decide, which is exactly why it never compresses.

It is the same tension as the squeeze between a camera and a decision, one level up. Something has to be thrown away, and the thing throwing it away does not know the future.

Splitting the difference

A useful trick, and one worth knowing because it shows up in the systems that work.

Carry two things forward rather than one. One part is computed the same way every time, and holds whatever reliably follows from the last state. The other is sampled, and holds whatever could still go either way. A ball's motion goes in the first. Whether the door opens goes in the second.

Keep only the deterministic part and the model cannot represent genuine uncertainty, so it confidently invents one future. Keep only the stochastic part and it struggles to remember anything for long, because noise gets injected at every step. PlaNet's argument for carrying both is that each one fails alone in a different direction.

Living with it

Nobody has removed the mismatch. What the field has instead is four ways of not being destroyed by it.

Train it on its own output. If the problem is that the model never practised recovering from its own mistakes, let it make some during training. Feed it its own predictions part of the time, and raise that fraction as it improves. The usual name for this is *scheduled sampling*.

Train the rollout, not the step. Instead of scoring one-step accuracy, score a whole imagined stretch against a whole real one, so the thing being optimised is the thing you will actually use.

Do not go far. Keep the rollouts short and start them from real states. Cheap, effective, and it gives up the part of the appeal that was infinite.

Look again. Predict a few steps, act on one, take a fresh observation, and start over. This is most of robotics, and it works because a real measurement resets the accumulated error to nothing.

None of these is a fix. They are all the same admission: a learned transition model is trustworthy over a horizon, and part of the engineering is knowing how long that horizon is.

Try it

1 A transition model reports a very low average one-step error. Why does that number not tell you what you want to know?

- a. It was probably measured wrong
- b. You will use it many steps at a time, and after the first step it is fed its own answer rather than the truth
- c. One-step errors are always understated
- d. Averages hide outliers

ANSWER b. You will use it many steps at a time, and after the first step it is fed its own answer rather than the truth. The measurement is honest. It is a measurement of a job the model will never be asked to do.

2 In the figure, the corrected line and the free-running line come from the same model with the same per-step bias. Why do they end up so far apart?

- a. The free-running one accumulates a larger bias
- b. One is given the true previous state at every step, so its mistakes never feed anything
- c. The corrected one uses a different model
- d. Random noise differs between the runs

ANSWER b. One is given the true previous state at every step, so its mistakes never feed anything. Correction resets the input to the truth every step, so error cannot compound. Nothing about the model changed between the two lines.

3 What is teacher forcing?

- a. Training on data labelled by a larger model
- b. Showing the model the true previous value at each training step, because that is what the recording contains
- c. Forcing the model to use a fixed learning rate
- d. Training only on the hardest examples

ANSWER b. Showing the model the true previous value at each training step, because that is what the recording contains. It makes training stable, and it also means the model is never once asked to recover from being slightly wrong, which is the only thing it will have to do later.

4 What does a fixed summary have to do that attention never has to do?

- a. Run faster
- b. Decide what is worth keeping, without knowing what will be needed later
- c. Store the whole sequence
- d. Handle actions

ANSWER b. Decide what is worth keeping, without knowing what will be needed later. That is the whole trade. Attention never decides, which is exactly why it never compresses and why its cost grows with length.

5 For a short sequence, which is cheaper per step?

- a. Always the summary
- b. Always attention
- c. Attention, until the sequence gets long enough to cross over
- d. They are identical

ANSWER c. Attention, until the sequence gets long enough to cross over. Updating a summary costs the same whatever the length, so it only wins once the sequence is long. Below the crossover, looking at everything is genuinely the cheaper option.

6 Why carry a deterministic part and a stochastic part in the state at the same time?

- a. To use more parameters
- b. Deterministic alone cannot represent real uncertainty; stochastic alone struggles to remember, because noise enters at every step
- c. To make the model differentiable
- d. To support larger batches

ANSWER b. Deterministic alone cannot represent real uncertainty; stochastic alone struggles to remember, because noise enters at every step. Each one fails alone, and in a different direction. A ball's motion belongs in the first; whether the door opens belongs in the second.

7 Replanning after every real observation helps a great deal. Why?

- a. It makes the model more accurate
- b. A fresh measurement resets the accumulated error to nothing
- c. It reduces the number of steps computed
- d. It removes the model's bias

ANSWER b. A fresh measurement resets the accumulated error to nothing. It does not touch the model at all. It just stops letting the error compound for very long before reality gets a say.

8 What do scheduled sampling, rollout-level training, short horizons and replanning have in common?

- a. They fix the mismatch
- b. They all make the model more accurate per step
- c. None removes the mismatch; each limits how much damage it does
- d. They only apply to recurrent models

ANSWER c. None removes the mismatch; each limits how much damage it does. The honest summary is that a learned transition model is trustworthy over some horizon, and part of the engineering is knowing how long that is.

FIG. 5.3 Eight questions on the step and the mismatch. The first three are the ones that change how you read a paper's headline number.

So far every step has been paid for in reality. The model is trained on recorded experience, checked against recorded experience, and corrected by it.

Thanks for reading. Chapter 6 is about what happens when you stop paying. The model gets good enough that an agent can do its learning inside it, and then you find out what the agent found in there.

Sources

Scheduled Sampling for Sequence Prediction with Recurrent Neural Networks

BENGIO ET AL., 2015 ↗

The mismatch named and attacked head on: let the model eat its own predictions during training, and raise the dose as it improves.

<https://arxiv.org/abs/1506.03099>

Sequence Level Training with Recurrent Neural Networks RANZATO ET AL., 2015 ↗

Score the whole rollout rather than the single step, so the thing being optimised is the thing you will actually run.

<https://arxiv.org/abs/1511.06732>

Learning Latent Dynamics for Planning from Pixels (PlaNet) HAFNER ET AL., 2018 ↗

The argument for carrying a deterministic part and a stochastic part together, because each one fails alone in a different direction.

<https://arxiv.org/abs/1811.04551>

Dream to Control (Dreamer) HAFNER ET AL., 2019 ↗

What a latent transition model is for once it works: long imagined rollouts that behaviour can be learned from.

<https://arxiv.org/abs/1912.01603>

Long Short-Term Memory HOCHREITER & SCHMIDHUBER, 1997 ↗

The fixed summary, made to hold on to things for longer than the gradient wanted it to. Paywalled.

<https://doi.org/10.1162/neco.1997.9.8.1735>

Attention Is All You Need VASWANI ET AL., 2017 ↗

The other answer: stop summarising, keep every step, and pay for it in how the cost grows with length.

<https://arxiv.org/abs/1706.03762>

Efficiently Modeling Long Sequences with Structured State Spaces (S4)

GU ET AL., 2021 ↗

The summary approach returning with better machinery, and the reason state-space models are back in the conversation.

<https://arxiv.org/abs/2111.00396>

Mamba: Linear-Time Sequence Modeling with Selective State Spaces GU & DAO, 2023



A summary that decides what to keep based on what it is looking at, which is the concession the fixed version could not make.

<https://arxiv.org/abs/2312.00752>

FIG. 5.4 Ordered for someone building on this rather than for historical completeness.